

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7
INHERITANCE DAN POLYMORPHISM (SISTEM
EKSPEDISI)



Disusun Oleh:

Glory Hardy Febriando Sianturi (105223013)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Program Layanan Ekspedisi ini adalah sebuah sistem simulasi logistik yang dirancang untuk menghitung ongkos kirim berdasarkan berbagai jenis layanan, yaitu Reguler, Express, dan Internasional. Tujuan utama dari program ini adalah mengimplementasikan konsep inti Pemrograman Berorientasi Objek (OOP), khususnya *Inheritance* (Pewarisan) dan *Polymorphism* (Polimorfisme).

Tantangan utama dalam pengembangan program ini adalah mendesain struktur hierarki kelas yang baik, di mana `LayananEkspedisi` bertindak sebagai *Superclass*, sementara layanan spesifik menjadi *Subclass*. Program ini menuntut penerapan *Method Overriding* untuk perhitungan tarif yang berbeda tiap layanan, *Method Overloading* untuk penambahan fitur diskon dan jarak, serta penggunaan teknik *Upcasting* dan *Downcasting* pada koleksi data (`ArrayList`) untuk memanipulasi objek secara dinamis.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	nomorResi	String	Menyimpan data nomor identitas pengiriman paket.
2	beratAktualKg	double	Menyimpan berat asli paket dalam satuan kilogram.
3	panjang, lebar, tinggi	double	Menyimpan dimensi paket (cm) untuk perhitungan volumetrik.
4	negaraTujuan	String	(Khusus Internasional) Menyimpan nama negara tujuan pengiriman.
5	nilaiBarangUSD	double	(Khusus Internasional) Menyimpan nilai deklarasi barang dalam mata uang USD untuk perhitungan pajak.
6	isMember	boolean	Penanda apakah klien merupakan member untuk mendapatkan diskon.
7	jarakKm	int	Menyimpan jarak pengiriman (km) untuk perhitungan <i>surcharge</i> .
8	totalPendapatan Perusahaan	double	Menampung hasil akumulasi seluruh ongkir dasar dari semua resi yang diproses.
9	listLayanan	ArrayList	Koleksi data untuk menyimpan berbagai objek subclass ekspedisi dalam satu wadah <i>superclass</i> (<i>Upcasting</i>).

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	LayananEkspedisi(...)	<i>Constructor</i>	Menginisialisasi state awal objek (nomor resi, berat, dimensi) saat paket didaftarkan.
2	hitungBeratEfektif()	<i>Functional</i>	Mengembalikan nilai double tertinggi antara berat aktual dan berat volumetrik paket.
3	cetakResi()	<i>Procedural</i>	Menampilkan output ke layar berupa nomor resi dan berat efektif paket.
4	hitungOngkir()	<i>Functional</i>	(Polymorphic) Menghitung dan mengembalikan nilai ongkir dasar sesuai tarif masing-masing layanan (<i>Overriding</i>).
5	hitungOngkir(boolea n, int)	<i>Functional</i>	(<i>Overloading</i>) Menghitung ongkir layanan Reguler dengan tambahan logika diskon member dan <i>surcharge</i> jarak.
6	klaimAsuransi(doubl e)	<i>Procedural</i>	Method spesifik Express untuk mengecek dan mencetak status klaim asuransi VIP/Standar.
7	cetakManifest()	<i>Procedural</i>	Method spesifik Internasional untuk mencetak data tujuan negara dan nilai deklarasi barang.

IV. Dokumentasi dan Pembahasan Code

4.1. Penjelasan Kode LayananEkspedisi.java (Superclass)

```

src > LayananEkspedisi.java > LayananEkspedisi
1 public class LayananEkspedisi {
2     // Atribut menggunakan protected agar bisa diturunkan
3     protected String nomorResi;
4     protected double beratAktualKg;
5     protected double panjang;
6     protected double lebar;
7     protected double tinggi;
8
9     // Konstruktor untuk menginisialisasi atribut
10    public LayananEkspedisi(String nomorResi, double beratAktualKg, double panjang, double lebar, double tinggi) {
11        this.nomorResi = nomorResi;
12        this.beratAktualKg = beratAktualKg;
13        this.panjang = panjang;
14        this.lebar = lebar;
15        this.tinggi = tinggi;
16    }
17
18    // Mengembalikan nilai tertinggi antara berat aktual dan volumetrik
19    public double hitungBeratEfektif() {
20        double beratVolumetrik = (panjang * lebar * tinggi) / 6000.0;
21        return Math.max(beratAktualKg, beratVolumetrik);
22    }
23
24    // Mencetak nomor resi dan berat efektif
25    public void cetakResi() {
26        System.out.println("Nomor Resi: " + nomorResi + " | Berat Efektif: " + hitungBeratEfektif());
27    }
28
29    // Metode polymorphic yang akan di-override
30    public double hitungOngkir() {
31        return 0.0;
32    }
33 }

```

Kelas ini bertindak sebagai kelas induk (*Superclass*).

- Access Modifier protected: Atribut seperti nomorResi, beratAktualKg, dll., menggunakan modifier protected. Tujuannya agar atribut-atribut ini bisa diakses langsung oleh kelas-kelas turunannya (subclass) tanpa harus membuat *getter* dan *setter*.
- Method hitungBeratEfektif(): Menggunakan fungsi matematika bawaan Java, yaitu Math.max(), untuk membandingkan beratAktualKg dengan beratVolumetrik. Fungsi ini akan otomatis mengembalikan nilai yang paling besar, sehingga kita tidak perlu menuliskan logika if-else secara manual.
- Method hitungOngkir(): Dibuat mengembalikan nilai 0.0. Method ini bertindak sebagai kerangka dasar (*default behavior*) yang nantinya wajib ditimpa (*di-override*) oleh masing-masing subclass sesuai tarif layanan mereka.

4.2. Penjelasan Kode LayananReguler.java (Subclass)

```

src > LayananReguler.java > LayananReguler
1 public class LayananReguler extends LayananEkspedisi {
2
3     public LayananReguler(String nomorResi, double beratAktualKg, double panjang,
4         super(nomorResi, beratAktualKg, panjang, lebar, tinggi);
5     }
6
7     @Override
8     public double hitungOngkir() {
9         // Tarif dasar Rp 15.000 per Kg (dari Berat Efektif)
10        return 15000 * hitungBeratEfektif();
11    }
12
13    // Overloading: Menghitung ongkir dengan diskon member dan surcharge jarak
14    public double hitungOngkir(boolean isMember, int jarakKm) {
15        double tarifDasar = hitungOngkir(); // Ambil tarif dasar
16
17        if (isMember) {
18            tarifDasar = tarifDasar - (tarifDasar * 0.10); // Diskon 10%
19        }
20
21        double surcharge = 500 * jarakKm; // Surcharge jarak jauh Rp 500 per km
22        return tarifDasar + surcharge;
23    }
24 }

```

Kelas ini mewarisi sifat dari LayananEkspedisi menggunakan *keyword* extends.

- Keyword `super()`: Di dalam konstruktor, pemanggilan `super(...)` digunakan untuk mengirimkan nilai parameter dari kelas LayananReguler ke konstruktor kelas induknya (LayananEkspedisi) agar atribut dasar bisa diinisialisasi.
- Method Overriding: Terdapat anotasi `@Override` pada `hitungOngkir()`. Ini menandakan bahwa kelas ini memodifikasi rumus ongkir bawaan dari superclass menjadi spesifik untuk layanan reguler, yaitu dikali 15000.
- Method Overloading: Kelas ini memiliki dua method `hitungOngkir`. Yang satu tanpa parameter, dan yang kedua memiliki parameter (`boolean isMember`, `int jarakKm`). Ini disebut *Overloading*, di mana kita membuat method dengan nama yang sama persis, tetapi memiliki fungsi dan parameter masukan yang berbeda (untuk mengakomodasi diskon dan *surcharge*).

4.3. Penjelasan Kode LayananExpress.java (Subclass)

```

src > LayananExpress.java > LayananExpress
1 public class LayananExpress extends LayananEkspedisi {
2
3     public LayananExpress(String nomorResi, double beratAktualKg, double panjang,
4         super(nomorResi, beratAktualKg, panjang, lebar, tinggi);
5     }
6
7     @Override
8     public double hitungOngkir() {
9         // Tarif dasar Rp 30.000 per Kg
10        return 30000 * hitungBeratEfektif();
11    }
12
13    // Metode spesifik klaim asuransi
14    public void klaimAsuransi(double nilaiBarang) {
15        if (nilaiBarang > 1000000) {
16            System.out.println("Klaim Asuransi VIP Rp" + nilaiBarang + " untuk re:");
17        } else {
18            System.out.println("Klaim Asuransi Standar diproses dalam 7 hari kerj:");
19        }
20    }
21 }

```

Sama seperti kelas sebelumnya, kelas ini adalah turunan dari LayananEkspedisi.

- **Overriding Tarif Khusus:** Kelas ini menimpa method `hitungOngkir()` dengan tarif pengali 30000.
- **Method Spesifik Subclass:** Terdapat method `klaimAsuransi(double nilaiBarang)`. Method ini adalah "ciri khas" dari Layanan Express dan tidak dimiliki oleh kelas induk maupun kelas reguler. Ini mendemonstrasikan bahwa sebuah subclass bebas memiliki perilaku (*behavior*) tambahannya sendiri di luar apa yang diwariskan kepadanya.

4.4. Penjelasan Kode LayananInternasional.java (Subclass)

```
src > LayananInternasional.java > LayananInternasional
1 public class LayananInternasional extends LayananEkspedisi {
2     private String negaraTujuan;
3     private double nilaiBarangUSD;
4
5     public LayananInternasional(String nomorResi, double beratAktualKg, double panjang, double lebar, double tinggi, double nilaiBarangUSD) {
6         super(nomorResi, beratAktualKg, panjang, lebar, tinggi);
7         this.negaraTujuan = negaraTujuan;
8         this.nilaiBarangUSD = nilaiBarangUSD;
9     }
10
11     @Override
12     public double hitungOngkir() {
13         double tarifDasar = 200000 * hitungBeratEfektif();
14         double pajak = 0.0;
15
16         // Pajak 20% jika nilai barang > 50 USD
17         if (nilaiBarangUSD > 50) {
18             pajak = 0.20 * tarifDasar;
19         }
20
21         return tarifDasar + pajak;
22     }
23
24     // Metode spesifik cetak manifest
25     public void cetakManifest() {
26         System.out.println("Manifest Internasional ke " + negaraTujuan + " - Deklarasi");
27     }
28 }
```

Kelas ini menangani pengiriman luar negeri dengan perhitungan yang lebih kompleks.

- **Penambahan Atribut Private:** Berbeda dengan kelas induk, kelas ini memiliki atribut tambahannya sendiri yaitu `negaraTujuan` dan `nilaiBarangUSD`.
- **Modifikasi Logika Overriding:** Pada method `hitungOngkir()`, *overriding* tidak hanya sekedar mengalikan berat dengan tarif dasar (200000), tetapi juga menambahkan logika percabangan `if` untuk mengecek apakah barang tersebut bernilai di atas 50 USD guna menghitung penambahan biaya pajak Bea Cukai sebesar 20%.

4.5 Penjelasan Kode Main.java (Main Class)

```
src > Main.java > ...
1  import java.util.ArrayList;
2
3  public class Main {
4      Run | Debug
5      public static void main(String[] args) {
6          // 1. Buat ArrayList (Upcasting)
7          ArrayList<LayananEkspedisi> listLayanan = new ArrayList<>();
8
9          // 2. Tambahkan data sesuai spesifikasi
10         listLayanan.add(new LayananReguler("REG-11", 2, 50, 50, 50));
11         listLayanan.add(new LayananExpress("EXP-22", 5, 10, 10, 10));
12         listLayanan.add(new LayananInternasional("INT-33", 3, 20, 20, 20, "Korea", 100));
13
14         // 3. Variabel total pendapatan
15         double totalPendapatanPerusahaan = 0.0;
16
17         System.out.println("=== DETAIL LAYANAN EKSPEDISI ===");
18
19         // 4. Perulangan menelusuri koleksi
20         for (LayananEkspedisi layanan : listLayanan) {
21             System.out.println("-----");
22
23             // Panggil cetakResi
24             layanan.cetakResi();
25
26             // Tambahkan ongkir dasar ke total pendapatan
27             totalPendapatanPerusahaan += layanan.hitungOngkir();
28
29             // 5. Gunakan instanceof dan Downcasting untuk method spesifik
30             if (layanan instanceof LayananReguler) {
31                 LayananReguler reg = (LayananReguler) layanan;
32                 System.out.println("Harga Layanan Reguler (Member + Jarak 25Km): Rp " + reg.hitungOngkir(true, 25));
33             }
34             else if (layanan instanceof LayananExpress) {
35                 LayananExpress exp = (LayananExpress) layanan;
36                 exp.klaimAsuransi(1500000);
37             }
38             else if (layanan instanceof LayananInternasional) {
39                 LayananInternasional intl = (LayananInternasional) layanan;
40                 intl.cetakManifest();
41             }
42         }
43
44         System.out.println("=====");
45         // 6. Cetak total pendapatan di akhir
46         System.out.println("Total Pendapatan Perusahaan (Ongkir Dasar): Rp " + totalPendapatanPerusahaan);
47     }
48 }
```

Ini adalah jantung dari program yang mengeksekusi semua logika OOP yang telah dibuat.

- Upcasting pada Koleksi Data: Baris `ArrayList<LayananEkspedisi> listLayanan = new ArrayList<>();` adalah contoh *Upcasting*. Kita membuat satu buah list bertipe *Superclass*, tetapi kita mengisinya dengan berbagai objek *Subclass* (Reguler, Express, Internasional). Ini membuat pengelolaan data menjadi sangat rapi dan dinamis.
- Polymorphism (Polimorfisme): Di dalam perulangan `for`, saat program memanggil `layanan.hitungOngkir()`, Java secara pintar akan melihat *wujud asli* dari objek tersebut (Dynamic Method Dispatch). Jika yang sedang dilooping adalah objek Reguler, maka ia memakai tarif 15.000, jika Express maka 30.000. Program tidak akan tertukar meskipun pemanggilannya menggunakan variabel referensi dari superclass.
- Downcasting dan instanceof: Karena objek-objek tersebut sudah "dibungkus" (*upcast*) menjadi `LayananEkspedisi`, mereka kehilangan akses ke method spesifik mereka (seperti `cetakManifest` atau `klaimAsuransi`). Untuk memanggilnya kembali, kita harus menggunakan `instanceof` untuk mengecek tipe aslinya, lalu melakukan *Downcasting* (misalnya `(LayananInternasional) layanan`) untuk membuka akses ke method spesifik tersebut dengan aman tanpa menyebabkan *error* sistem.

V. Kesimpulan

Dari hasil perancangan dan implementasi program Sistem Layanan Ekspedisi ini, dapat ditarik beberapa kesimpulan:

1. Efektivitas Inheritance: Penggunaan *superclass* `LayananEkspedisi` menghilangkan duplikasi kode yang berulang (seperti deklarasi atribut dan method pencetak resi), sehingga kode menjadi lebih rapi dan modular.
2. Fleksibilitas Polymorphism & Overloading: Sistem dengan mudah menangani perhitungan biaya yang berbeda-beda (*overriding*) dan variasi parameter untuk member/jarak tempuh (*overloading*) tanpa merusak fungsi yang sudah ada.
3. Kekuatan Casting pada Koleksi Data: *Upcasting* memungkinkan perusahaan memproses ribuan data dari berbagai tipe layanan di dalam satu perulangan (`for`) yang sama. Sementara *Downcasting* yang diiringi pengamanan `instanceof` memungkinkan akses aman ke fitur eksklusif tiap layanan (seperti cetak manifest internasional atau klaim asuransi) tanpa error `ClassCastException`.

VI. Daftar Pustaka

- Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education.
- Oracle. (2026). *Java SE Documentation: Object-Oriented Programming Concepts*. [Online]. Tersedia di: <https://docs.oracle.com/en/java/>
- Schildt, H. (2018). *Java: The Complete Reference*. McGraw-Hill Education.